# IP Hash and Sticky Sessions

*pin a user to one server, and what it costs*

Watch the 4-min video

**In one line:** sticky sessions (session affinity) send every request from one client to the same backend, by IP hash or by cookie, so in-memory state keeps working; the price is uneven load, lost sessions when a server dies, and harder deploys and scaling, so the real fix is to keep servers stateless and put state outside.

## 1 The empty cart

A user adds three items to their cart. The next request lands on a different server, and the cart is empty. The old app kept sessions in each server's memory, so only server 1 knows about the cart. The quick fix is to make the load balancer always send that user to the same server. That's a **sticky session**.

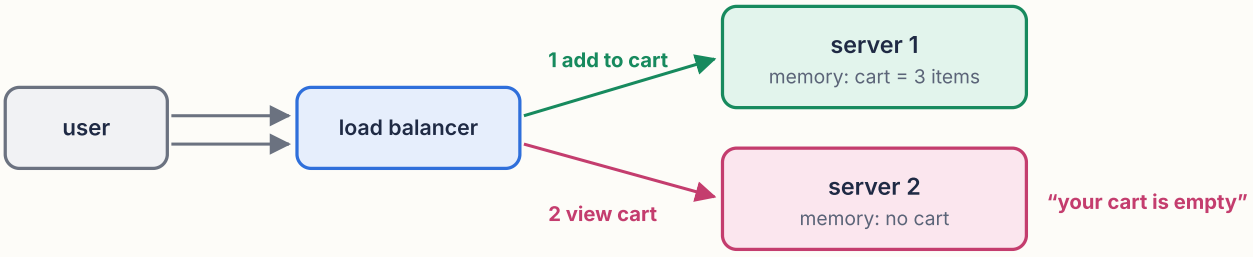



**Figure 1.** With sessions in server memory, a round-robin load balancer sends the second request to a server that has never seen the cart.

## 2 Session affinity

**Session affinity**, or stickiness, means the load balancer routes all requests from one client to the same backend. You need it when a server keeps **per-user state locally**: an in-memory session, a shopping cart, a multi-step upload, or a WebSocket's context. There are two common ways to do it.

## 3 IP hash, and cookie stickiness

**IP hash** hashes the client's IP address and picks a backend from the hash. Same IP, same server, and the balancer keeps no state. But many users can share one public IP behind a corporate proxy or a mobile carrier's NAT (carrier-grade NAT puts thousands of phones behind a few addresses), which creates hot servers. And a phone that switches from Wi-Fi to 4G gets a new IP and lands somewhere else.

**Cookie-based stickiness** is more precise. On the first response the load balancer sets a cookie naming the backend (AWS ALB's `AWSALB`, an NGINX `sticky cookie`), or the app sets its own session cookie that the balancer reads (application-based stickiness). Later requests carry the cookie and go back to the same server, usually until the cookie expires.




**Figure 2.** The two ways to stick: by source IP (stateless, any protocol, but coarse) or by cookie (per browser, but HTTP only).

## 4 The bill

- **Uneven load.** A few heavy users pin themselves to one server, and new servers only get *new* users, so scaling out helps slowly.
- **Failover loses state.** If that server dies, every session on it is gone: users are logged out or lose their carts.
- **Harder deploys and scale-in.** You must drain sessions before removing a server, so rolling deploys and autoscaling take much longer.
- **Hidden coupling.** The app quietly depends on the load balancer's behaviour; change one setting and users see bugs.

**WORKED EXAMPLE: A LEGACY APP WITH IN-MEMORY SESSIONS**

The short-term fix is cookie-based stickiness, so the app keeps working behind the load balancer. The real fix is to move sessions out:

- into **Redis** (key `session:abc123` with a TTL), or a database table;
- or into a **signed token** (a JWT or a signed cookie) that the client carries, so no server needs to look anything up.

Then any server can handle any request, stickiness can be turned off, and a server can die or be replaced without anyone losing a cart.

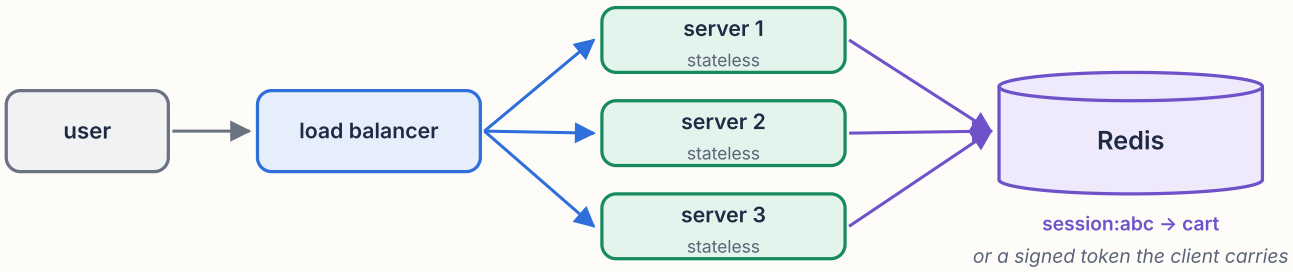



**Figure 3.** The fix: stateless servers and an external session store. The load balancer can send each request anywhere.

## 5 When stickiness is fine

Stickiness isn't always bad. It can be a **performance optimisation**: a server's local cache warms up for the users it sees, so pinning them helps, as long as losing that pinning is harmless. Long-lived WebSocket

connections are naturally pinned anyway. The rule: **affinity may help speed, but correctness must never depend on it.**

**COMMON MISTAKES**

- Using stickiness to hide in-memory sessions, then losing sessions on every deploy.
- IP hash for a mobile or enterprise audience, with many users behind one IP.
- Forgetting a cookie expiry, so users stay pinned to a server long after it's overloaded.
- Storing whole carts or large objects in a JWT: tokens travel on every request and can't be revoked easily.

## 6 Key terms

**Sticky session**: all requests from one client go to one backend

**Session affinity**: another name for stickiness

**IP hash**: choose the backend from a hash of the client IP

**Carrier-grade NAT**: many phones sharing a few public IPs

**Duration-based cookie**: a balancer-made cookie naming the backend

**Stateless server**: keeps no per-user state between requests

**Session store**: external place for sessions, e.g. Redis

**Signed token**: state the client carries, protected by a signature

## 7 Interview questions

**Q1. Why are sticky sessions problematic?** BASIC
They cause uneven load, lose user sessions when a server fails, and make deploys and autoscaling harder; IP hash also breaks with shared or changing IP addresses.

**Q2. What's the better approach?** BASIC
Store session state externally, in Redis, a database, or a signed token the client carries, so any server can handle any request and stickiness isn't needed.

**Q3. IP hash or cookie stickiness: which is more precise?** BASIC
Cookie stickiness: it identifies the browser rather than the network, so users behind one proxy spread out and a phone changing networks keeps its server. IP hash works for non-HTTP protocols, though.

**Q4. After a deploy, many users are logged out. The app uses sticky sessions. Why?** SCENARIO
Sessions lived in the memory of the servers that were replaced. Move sessions to a shared store or tokens, and meanwhile drain servers (wait for sessions to expire) before removing them.

**Q5. You doubled your servers, but the old ones are still at 90% CPU. Why?** SCENARIO
Stickiness keeps existing users on the old servers; only new users reach the new ones. Shorten the stickiness duration, or remove stickiness by externalising state.

**Q6. When is stickiness a good idea?** TRICKY
When it's only an optimisation, such as warm per-user caches, or for long-lived connections like WebSockets, and the system stays correct if a user moves to another server.

Next: SD-G-007 · Health Checks backendroadmap.com